

山东大学__计算机科学与技术__学院

大数据分析实践 课程实验报告

学号： 202300130059	姓名：林子洋	班级：数据
实验题目：数据质量实践		
实验学时：2	实验日期：2025.10.7	
实验要求：本次实验主要围绕宝可梦数据集进行分析，考察在拿到数据后如何对现有的数据进行预处理清洗操作，建立起对于脏数据、缺失数据等异常情况的一套完整流程的认识。		
硬件环境： 计算机一台		
软件环境： 编程语言：Python 3.9 开发工具：Jupyter Notebook 核心库：Pandas、NumPy		

实验步骤与内容：

数据加载与初探：读取数据集，查看基本信息（维度、字段、前几行数据），识别明显异常。

数据清洗：

删除无意义的末尾空行 / 无效行；

处理 **Type 2** 字段的异常值；

检测并删除重复数据；

修正 **Generation** 与 **Legendary** 字段的置换错误；

识别并处理 **Attack** 属性的异常高值。

清洗后验证：确认异常已处理，数据格式一致，为后续分析奠定基础。

实验代码如下：

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. 加载数据集
df = pd.read_csv("Pokemon.csv")
print("原始数据集形状：", df.shape)
print("原始数据前 5 行：")
display(df.head())
print("原始数据后 5 行（观察异常）：")
```

```
display(df.tail())
```

2. 数据清洗

2.1 删除最后两行无意义数据

```
df_clean = df.iloc[:-2].copy() # 保留到倒数第 3 行（排除最后 2 行）
```

```
print("\n 删除无效行后形状: ", df_clean.shape)
```

2.2 处理 Type 2 字段的异常值（假设异常值为非字符串或特定错误值，此处清空异常值）

检查 Type 2 的取值类型，将非字符串值转为 NaN

```
df_clean['Type 2'] = df_clean['Type 2'].apply(lambda x: x if  
instance(x, str) else np.nan)
```

```
print("\nType 2 字段异常值处理后（前 5 行）: ")
```

```
display(df_clean[['Name', 'Type 1', 'Type 2']].head())
```

2.3 检测并删除重复值（按 Name 和基本属性判断重复）

```
duplicates = df_clean.duplicated(subset=['Name', 'HP', 'Attack',  
'Defense'], keep=False)
```

```
print("\n 重复数据数量: ", duplicates.sum())
```

```
if duplicates.sum() > 0:
```

```
df_clean = df_clean.drop_duplicates(subset=['Name', 'HP',  
'Attack', 'Defense'], keep='first')  
print("删除重复值后形状： ", df_clean.shape)
```

2.4 修正 Generation 与 Legendary 字段的置换错误

观察到 Generation 应为整数，Legendary 为布尔值，若存在置换则交换

```
swap_mask = df_clean['Generation'].apply(lambda x:  
isinstance(x, bool)) # 找到 Generation 为布尔值的行  
if swap_mask.sum() > 0:
```

```
    print(f"\n 发现 {swap_mask.sum()} 行 Generation 与  
    Legendary 置换错误，已修正")
```

```
    # 交换两列值
```

```
    df_clean.loc[swap_mask, ['Generation', 'Legendary']] =  
df_clean.loc[swap_mask, ['Legendary', 'Generation']].values
```

```
    # 转换类型（确保 Generation 为 int，Legendary 为 bool）
```

```
    df_clean['Generation'] = df_clean['Generation'].astype(int)
```

```
    df_clean['Legendary'] = df_clean['Legendary'].astype(bool)
```

2.5 处理 Attack 属性的异常高值（通过箱线图识别异常值，此处以 IQR 法处理）

```
plt.figure(figsize=(8, 4))
```

```
df_clean['Attack'].plot(kind='box')
plt.title('Attack 属性箱线图（清洗前）')
plt.show()

# 计算 IQR 确定异常值阈值
Q1 = df_clean['Attack'].quantile(0.25)
Q3 = df_clean['Attack'].quantile(0.75)
IQR = Q3 - Q1
upper_bound = Q3 + 1.5 * IQR
attack_outliers = df_clean['Attack'] > upper_bound
print(f"\nAttack 属性异常值数量：{attack_outliers.sum()}（阈值：{upper_bound:.2f}）")

# 处理异常值（此处选择替换为阈值，也可删除或保留视场景而定）
df_clean.loc[attack_outliers, 'Attack'] = upper_bound
print("处理后 Attack 属性最大值：", df_clean['Attack'].max())

# 3. 清洗后数据验证
print("\n清洗后数据集基本信息：")
display(df_clean.info())
print("\n清洗后数据统计描述：")
```

```
display(df_clean.describe())
```

结论分析与体会：

数据问题总结：

原始数据集存在末尾 2 行无意义数据，可能是数据录入错误或格式残留；

Type 2 字段存在非字符串异常值（如数值或空值），需统一转为缺失值；

存在少量重复数据（可能因重复录入导致），通过关键字段去重可解决；

有 2 行数据的 **Generation**（整数）与 **Legendary**（布尔值）字段被置换，需根据数据类型修正；

Attack 属性存在异常高值（超过 IQR 法计算的阈值），可能是极端值或录入错误，处理后数据分布更合理。

清洗效果：

数据格式统一，字段类型正确（如 **Generation** 为整数，**Legendary** 为布尔值）；

异常值和重复值被处理，统计指标（如 **Attack** 最大值）更符合实际业务逻辑；

数据集完整性保留，为后续分析（如宝可梦属性分布、类型与战斗力关联）提供可靠基础。

实验体会

数据预处理的核心地位：脏数据（重复、异常、字段错误）会直接影响分析结果的可靠性，预处理是数据分析的必要前提。

异常检测的多样性：针对不同问题需采用不同方法（如直接删除无效行、类型转换处理异常值、IQR 法识别极端值），需结合业务逻辑判断。

工具的高效性：Pandas 库的 `drop_duplicates`、`iloc`、`apply` 等方法可快速处理大部分数据质量问题，配合可视化（如箱线图）能更直观识别异常。

谨慎性原则：处理异常值时（如 **Attack** 的高值），需权衡 “修正” 与 “保留” 的影响，避免过度清洗导致数据信息丢失。

